

교과목명	융합UI실습																			
학 번	2601110288	이 름	박수용																	
능력단위	인터페이스 구현 (20001020212_19V5)	능력단위 요소	인터페이스 설계서 확인하기	평가방법	평가자 체크리스트															
			인터페이스 기능 구현하기																	
			인터페이스 구현 검증하기																	
요구사항																				
시뮬레이터를 연결하여 아래의 요구사항을 만족하는 GUI프로그램을 작성하시오.																				
1. 자동운전 [시작]과 [정지] 버튼으로 자동운전 공정을 진행하시오.																				
2. 리프트센서에 의해 물건이 올려지면 자동으로 실린더가 전/후진 동작을 수행되어야 함.																				
3. 리프트센서와 각 실린더의 동작 조건을 자세히 설명하시오.																				
첨부내용 : 각 기능별 함수코드와 설명, 결과 스크린샷																				
평가문항(수행내용)																				
1. 과제 개요																				
본 과제는 Mitsubishi PLC 시뮬레이터와 C# Windows Forms를 연동하여 자동운전 공정을 제어하는 GUI 프로그램을 구현하는 것을 목표로 하였습니다. ACTMULTILIB_K 라이브러리의 ActEasyIF 클래스를 활용하여 PLC 디바이스 주소를 읽고 쓰는 방식으로 실린더 제어를 수행하였습니다.																				
2. 요구사항 분석																				
<table><tr><th>번호</th><th>요구사항</th><th>구현 방법</th></tr><tr><td>①</td><td>자동운전 [시작] / [정지] 버튼 구현</td><td>btnStart_Click: 타이머 ON / btnStop_Click: 타이머 OFF + Close()</td></tr><tr><td>②</td><td>리프트센서 감지 시 실린더 자동 전/후진</td><td>timer1.Tick에서 X0 센서값 읽기 후 비트마스킹으로 조건 판단</td></tr><tr><td>③</td><td>동작 조건 자세히 설명</td><td>리프트센서 A(XA=0x0400), B(XB=0x0800) 기준 B/C실린더 제어 조건 명시</td></tr><tr><td>④</td><td>기능별 함수 코드와 설명 첨부</td><td>각 이벤트 핸들러별 코드 표 형태로 첨부</td></tr></table>						번호	요구사항	구현 방법	①	자동운전 [시작] / [정지] 버튼 구현	btnStart_Click: 타이머 ON / btnStop_Click: 타이머 OFF + Close()	②	리프트센서 감지 시 실린더 자동 전/후진	timer1.Tick에서 X0 센서값 읽기 후 비트마스킹으로 조건 판단	③	동작 조건 자세히 설명	리프트센서 A(XA=0x0400), B(XB=0x0800) 기준 B/C실린더 제어 조건 명시	④	기능별 함수 코드와 설명 첨부	각 이벤트 핸들러별 코드 표 형태로 첨부
번호	요구사항	구현 방법																		
①	자동운전 [시작] / [정지] 버튼 구현	btnStart_Click: 타이머 ON / btnStop_Click: 타이머 OFF + Close()																		
②	리프트센서 감지 시 실린더 자동 전/후진	timer1.Tick에서 X0 센서값 읽기 후 비트마스킹으로 조건 판단																		
③	동작 조건 자세히 설명	리프트센서 A(XA=0x0400), B(XB=0x0800) 기준 B/C실린더 제어 조건 명시																		
④	기능별 함수 코드와 설명 첨부	각 이벤트 핸들러별 코드 표 형태로 첨부																		

3. 디바이스 주소 구성

3.1 입력 디바이스 (X: 센서)

디바이스	비트 마스크	역할	설명
XA	0x0400	리프트센서 A	물건이 리프트A에 올라지면 ON
XB	0x0800	리프트센서 B	물건이 리프트B에 올라지면 ON

3.2 출력 디바이스 (Y: 실린더 제어)

디바이스	비트 마스크	역할	설명
Y01	0x0002	B실린더 전진	B실린더를 전진 방향으로 구동
Y02	0x0004	B실린더 후진	B실린더를 후진 방향으로 구동
Y03	0x0008	C실린더 전진	C실린더를 전진 방향으로 구동
Y04	0x0010	C실린더 후진	C실린더를 후진 방향으로 구동

4. 리프트센서와 실린더 동작 조건

4.1 B실린더 동작 조건 (리프트센서 A 기준)

조건	동작	설명
$XA(0x400) == 1$	B실린더 전진 (Y01 ON)	리프트센서 A에 물건이 감지되면 Y01을 켜고 Y02를 끄
$XA(0x400) == 0 + fb == 1$	B실린더 후진 (Y02 ON)	물건이 내려가고, 직전에 동작 중이었을 때 Y02를 켜고 Y01을 끄. 이후 $fb = 0$ 으로 초기화

4.2 C실린더 동작 조건 (리프트센서 B 기준)

조건	동작	설명
$XB(0x800) == 1$	C실린더 전진 (Y03 ON)	리프트센서 B에 물건이 감지되면 Y03을 켜고 Y04를 끄
$XB(0x800) == 0 + fb1 == 1$	C실린더 후진 (Y04 ON)	물건이 내려가고, 직전에 동작 중이었을 때 Y04를 켜고 Y03을 끄. 이후 $fb1 = 0$ 으로 초기화

4.3 자동운전 전체 동작 흐름

단계	동작	조건
①	연결 버튼 클릭	ActEasyIF.Open() 호출 → 성공 시 시작/정지 버튼 활성화
②	시작 버튼 클릭	타이머 ON → timer1_Tick 반복 실행 시작
③	리프트센서 A 감지	XA(0x400) = 1 → B실린더 전진(Y01 ON, Y02 OFF)
④	리프트센서 A 해제	XA(0x400) = 0, fb=1 → B실린더 후진(Y02 ON, Y01 OFF)
⑤	리프트센서 B 감지	XB(0x800) = 1 → C실린더 전진(Y03 ON, Y04 OFF)
⑥	리프트센서 B 해제	XB(0x800) = 0, fb1=1 → C실린더 후진(Y04 ON, Y03 OFF)
⑦	정지 버튼 클릭	타이머 OFF → Close() → 버튼 상태 초기화

1. 5. 기능별 함수 코드 및 설명

5.1 Form1_Load – 초기 설정

프로그램 시작 시 딱 한 번 실행되며, PLC 논리 스테이션 번호를 설정하고 시작/정지 버튼을 비활성화합니다.

```
private void Form1_Load(object sender, EventArgs e)
{
    control.ActLogicalStationNumber = 1; // 반드시 Open() 전에 설정
    btnStart.Enabled = false;           // 연결 전 시작 버튼 비활성화
    btnStop.Enabled = false;            // 연결 전 정지 버튼 비활성화
}
```

5.2 btnConnect_Click – 연결 버튼

PLC 시뮬레이터에 연결을 시도합니다. Open() 반환값이 0이면 성공이며, 시작/정지 버튼을 활성화하고 연결 버튼을 비활성화하여 중복 연결을 방지합니다.

```
private void btnConnect_Click(object sender, EventArgs e)
{
    if (control.Open() == 0)           // 0 = 연결 성공
    {
        MessageBox.Show("연결되었습니다.");
        btnStart.Enabled = true;       // 시작 버튼 활성화
        btnStop.Enabled = true;        // 정지 버튼 활성화
        btnConnect.Enabled = false;    // 중복 연결 방지
    }
    else { MessageBox.Show("연결 실패하였습니다."); }
}
```

5.3 btnStart_Click – 시작 버튼

타이머를 활성화하여 자동운전 공정을 시작합니다. 타이머가 켜지면 timer1_Tick이 설정된 간격마다 자동으로 반복 실행됩니다.

```
private void btnStart_Click(object sender, EventArgs e)
{
    timer1.Enabled = true;           // 타이머 ON → 자동운전 시작
    MessageBox.Show("자동운전 시작");
}
```

5.4 btnStop_Click – 정지 버튼

타이머를 비활성화하고 PLC 연결을 해제합니다. 이후 버튼 상태를 초기화하여 재연결이 가능한 상태로 복원합니다.

```
private void btnStop_Click(object sender, EventArgs e)
{
    timer1.Enabled = false;         // 타이머 OFF → 자동운전 중단
    control.Close();                // PLC 연결 해제
    btnConnect.Enabled = true;      // 재연결 가능하도록 활성화
    btnStart.Enabled = false;       // 시작 버튼 비활성화
    btnStop.Enabled = false;        // 정지 버튼 비활성화
    MessageBox.Show("자동운전 정지");
}
```

5.5 timer1_Tick – 자동운전 핵심 로직

타이머 틱마다 실행되며, X0 주소에서 센서값을 읽어 리프트센서 상태에 따라 B실린더와 C실린더를 자동으로 제어합니다. 비트마스킹을 이용하여 특정 비트만 켜고 끄는 방식으로 출력 충돌을 방지합니다.

```
private void timer1_Tick(object sender, EventArgs e)
{
    // ① X0 주소에서 1워드(16비트) 읽기 → 모든 센서 상태 수신
    control.ReadDeviceBlock2("X0", 1, out sensor);
}
```

```
// ② B실린더 제어 (리프트센서 A: XA = 비트 0x400)
```

```
if (((int)sensor & 0x0400) != 0)
```

```
{ // 센서 ON → Y01(전진) 켜기, Y02(후진) 끄기
```

```
    control.ReadDeviceBlock2("Y0", 1, out Bforward);
```

```
    Bforward = (short)((Bforward | 0x02) & ~0x04);
```

```
    control.WriteDeviceBlock2("Y0", 1, ref Bforward);
```

```
    fb = 1;
```

```
}
```

```
else if (((int)sensor & 0x0400) == 0 && fb == 1)
```

```
{ // 센서 OFF + 동작중 → Y02(후진) 켜기, Y01(전진) 끄기
```

```
    control.ReadDeviceBlock2("Y0", 1, out Bforward);
```

```
    Bforward = (short)((Bforward | 0x04) & ~0x02);
```

```
    control.WriteDeviceBlock2("Y0", 1, ref Bforward);
```

```
    fb = 0;
```

```
}
```

```
// ③ C실린더 제어 (리프트센서 B: XB = 비트 0x800)
```

```
if (((int)sensor & 0x0800) != 0)
```

```
{ // 센서 ON → Y03(전진) 켜기, Y04(후진) 끄기
```

```
    control.ReadDeviceBlock2("Y0", 1, out Cforward);
```

```
    Cforward = (short)((Cforward | 0x08) & ~0x10);
```

```
    control.WriteDeviceBlock2("Y0", 1, ref Cforward);
```

```
    fb1 = 1;
```

```
}
```

```
else if (((int)sensor & 0x0800) == 0 && fb1 == 1)
```

```
{ // 센서 OFF + 동작중 → Y04(후진) 켜기, Y03(전진) 끄기
```

```
    control.ReadDeviceBlock2("Y0", 1, out Cforward);
```

```
    Cforward = (short)((Cforward | 0x10) & ~0x08);
```

```
    control.WriteDeviceBlock2("Y0", 1, ref Cforward);
```

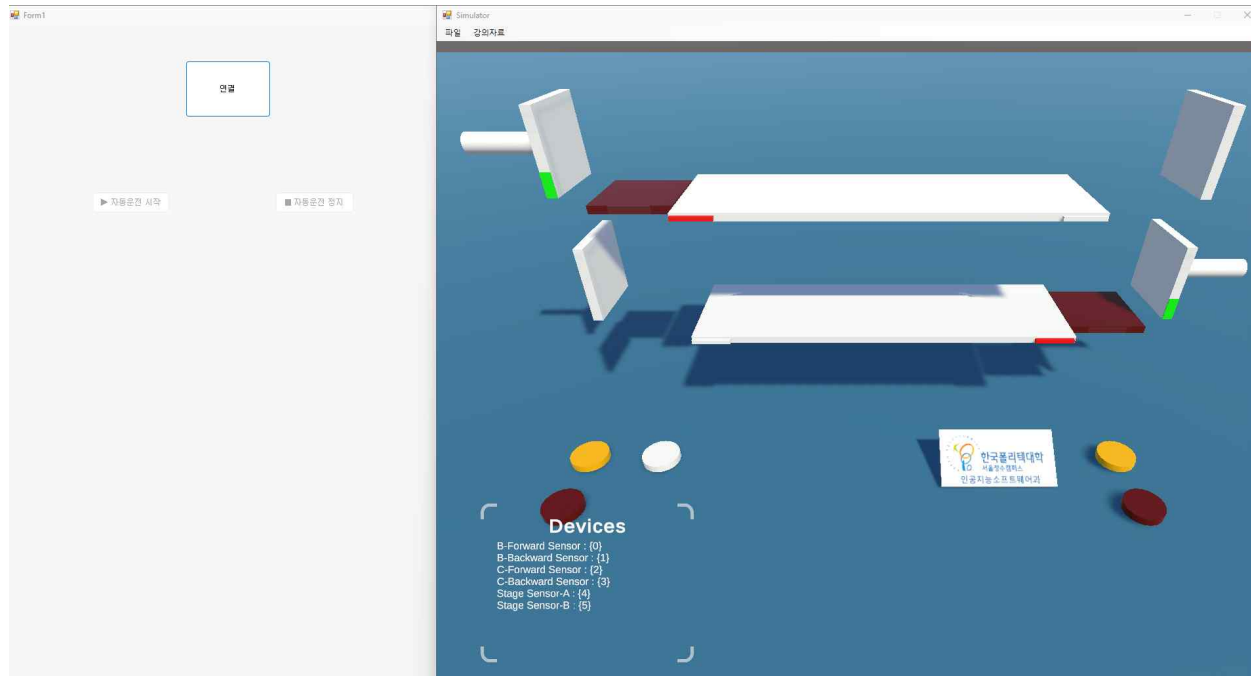
```
    fb1 = 0;
```

```
}
```

```
}
```

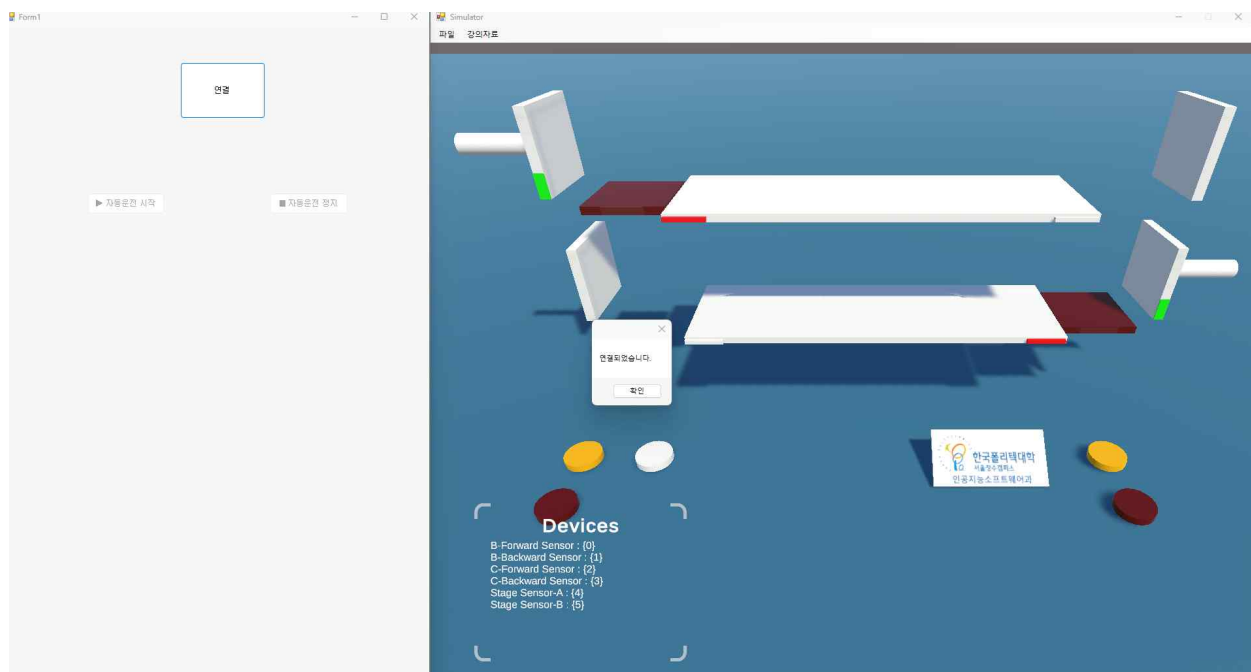
6. 과제 스크린 샷

1. 프로그램 초기 실행 화면



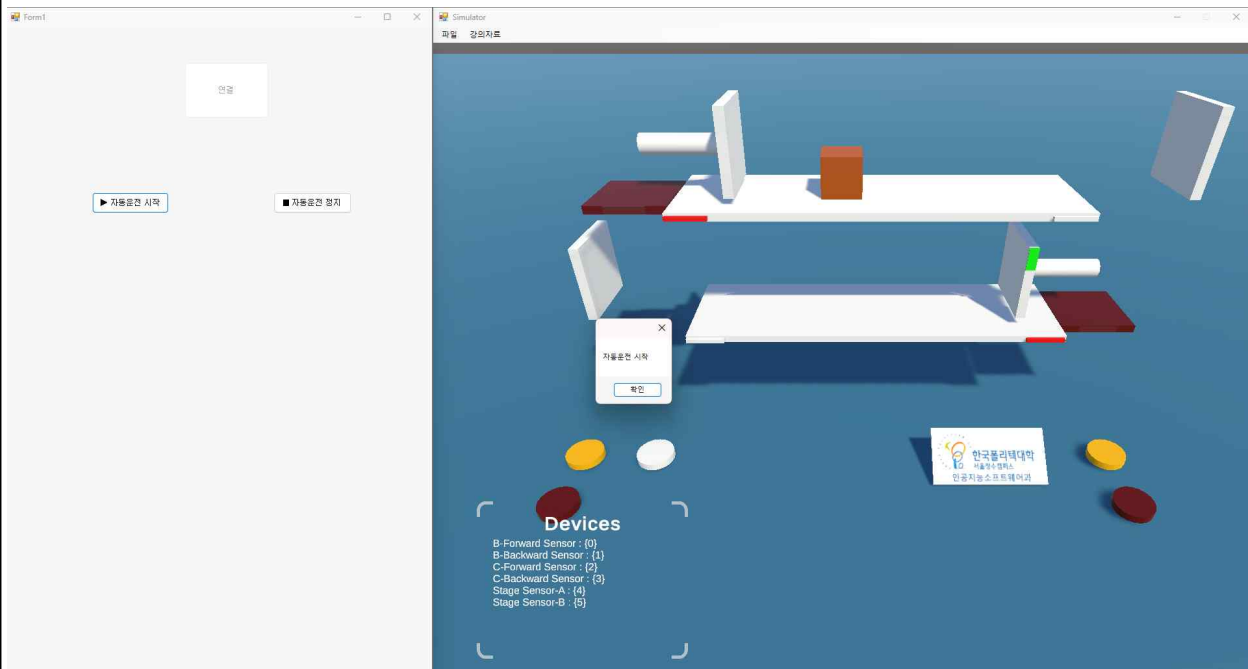
프로그램 실행 시 Form1_Load가 호출되어 btnStart.Enabled = false, btnStop.Enabled = false로 설정됩니다. 이에 따라 연결 버튼만 활성화된 상태로 시작되며, PLC 연결 전 시작·정지 버튼을 클릭할 수 없도록 안전하게 처리하였습니다.

2. PLC 시뮬레이터 연결 성공 화면



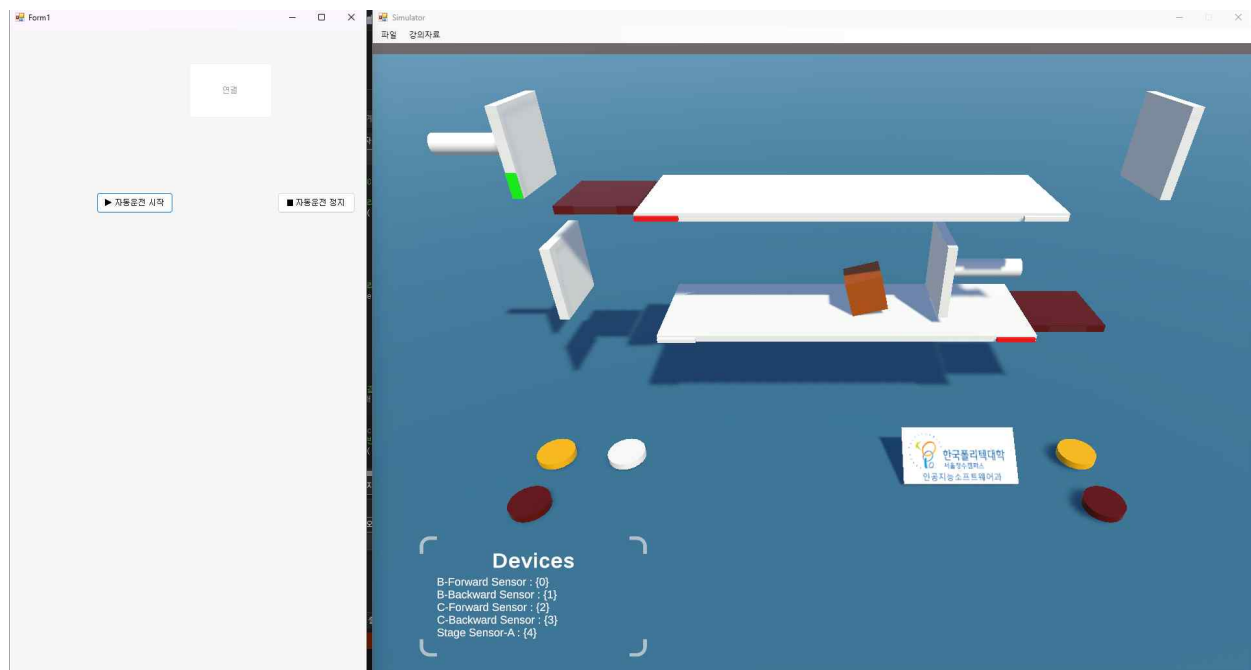
연결 버튼 클릭 시 btnConnect_Click이 실행되어 control.Open()을 호출합니다. 반환값이 0(성공)이면 "연결되었습니다." 메시지가 출력되며, 시작·정지 버튼이 활성화되고 연결 버튼은 비활성화되어 중복 연결을 방지합니다.

3. 자동운전 시작 화면



시작 버튼 클릭 시 btnStart_Click이 실행되어 timer1.Enabled = true로 타이머가 활성화됩니다. "자동운전 시작" 메시지 출력과 함께 timer1_Tick이 설정된 간격마다 반복 실행되기 시작하며, 시뮬레이터에서 리프트센서 A(Stage Sensor-A)에 물건이 올려진 것을 확인할 수 있습니다.

4. 실린더 자동 동작 화면



timer1_Tick 실행 중 리프트센서 B(Stage Sensor-B)에 물건이 감지되어 XB(0x0800) 비트가 ON 상태가 되면, C실린더 전진 명령(Y03 ON, Y04 OFF)이 자동으로 수행됩니다.

비트마스킹 연산 $Cforward = (short)((Cforward | 0x08) \& \sim 0x10)$ 이 적용되어 C실린더가 전진 동작하는 것을 시뮬레이터 화면에서 확인할 수 있습니다.

7. 결과 및 고찰

본 과제를 통해 C# Windows Forms와 PLC 시뮬레이터를 ACTMULTILIB_K 라이브러리로 연동하는 방법을 직접 구현하였습니다. 특히 비트마스킹 기법(OR 연산으로 특정 비트 켜기, AND NOT 연산으로 특정 비트 끄기)을 활용하여 여러 출력 디바이스를 하나의 워드 변수로 효율적으로 제어할 수 있었습니다.

또한 fb, fb1 플래그 변수를 활용하여 센서가 OFF가 되는 순간 정확히 한 번만 후진 명령이 실행되도록 설계함으로써, 타이머가 반복 실행되더라도 중복 명령이 전송되지 않도록 처리하였습니다.

연결 버튼, 시작 버튼, 정지 버튼의 활성화/비활성화 상태 관리를 통해 잘못된 순서로 버튼을 누르는 경우를 사전에 방지하였으며, 이를 통해 보다 안정적인 GUI 프로그램을 구현할 수 있었습니다.